

Trabalho Prático – Agente Inteligente em Labirinto

Esdras A. Ávila, Pedro H. E. Guedes, Samuel C. O. Aguiar

Universidade Federal de Ouro Preto (UFOP) – João Monlevade – MG – Brasil

{esdras.avila, pedro.esteves, samuell.aguiar}@aluno.ufop.edu.br

1. Introdução

Este trabalho prático consiste no desenvolvimento e na análise de um agente inteligente projetado para resolver labirintos sob três condições distintas de complexidade: mapa totalmente conhecido, otimização de rotas com múltiplos pontos de coleta e exploração de ambiente desconhecido. Em vez de criar três programas isolados, o grande desafio aqui é utilizar exatamente o mesmo domínio discreto para testar e comparar três filosofias de projeto de IA completamente diferentes: a busca clássica, a busca local e a busca online.

1.1. Busca Clássica

Na primeira etapa do trabalho (Busca Clássica), o foco está no planejamento offline. O agente recebe o mapa completo, calcula a rota ótima até o objetivo antes de realizar qualquer ação e executa o percurso.

1.2. Busca Local

Na segunda etapa (Busca Local), o problema se transforma em uma otimização combinatória análoga ao Problema do Caixeiro Viajante. O agente precisa descobrir qual é a melhor ordem de visita para coletar vários pontos obrigatórios antes de ir para a saída (B), testando algoritmos que refinam essa sequência a cada iteração.

1.3. Busca Online

Nesta etapa, o agente não possui acesso ao mapa completo do labirinto e deve explorá-lo progressivamente enquanto se move em direção ao objetivo. O desafio central é que, diferentemente da busca clássica (onde o agente planeja com o mapa completo), na busca online o agente deve descobrir o mapa enquanto age, seguindo o ciclo: perceber → atualizar mapa interno → planejar → agir

O objetivo deste relatório é documentar o processo de modelagem formal do agente com base na estrutura PEAS, detalhar as escolhas de implementação em Python e avaliar o desempenho dos algoritmos por meio das métricas exigidas. Ao analisar dados com o custo do caminho, nós expandidos e a razão entre o custo online e offline, faremos uma avaliação crítica sobre as vantagens e limitações de cada abordagem.

2. Modelagem PEAS

2.1. Performance

O comportamento do agente é avaliado por uma função de desempenho, baseada em um sistema de penalidades. Como todos os parâmetros possuem valores negativos, o objetivo do agente é maximizar o valor de J, tentando aproximá-lo de zero através de escolhas eficientes. A equação que calcula o desempenho do agente é dada por:

$$J = -(0,15 \times \text{custo_caminho} + 0,15 \times \text{nós_expandidos} + 0,3 \times \text{mov_inválidos} + 0,4 \times \text{células_revisitadas})$$

Os pesos escolhidos para cada parâmetro foram definidos para equilibrar os objetivos do projeto:

- **Custo do Caminho (0,15):** Pune o tamanho do caminho final, incentivando o agente a encontrar rotas mais curtas.
- **Nós Expandidos (0,15):** Penaliza o uso excessivo de memória, evitando que o algoritmo gaste processamento desnecessário na busca.

- **Movimentos Inválidos (0,3):** Aplica uma penalidade alta se o agente tentar andar em direção a uma parede ou para fora dos limites do mapa.
- **Células Revisitadas (0,4):** É a maior penalidade do sistema, criada para evitar que o agente fique andando em círculos, principalmente quando estiver explorando o mapa desconhecido.

2.2. Environment (Ambiente)

O ambiente do problema é um labirinto em formato de grade bidimensional, discreto e estático. Os elementos que compõem o mapa são:

- **Células livres:** Espaços vazios por onde o agente pode se movimentar.
- **Paredes:** Obstáculos intransitáveis representados pelo caractere #, que bloqueiam a passagem.
- **Posição inicial:** O ponto de partida do agente, indicado pela letra A.
- **Objetivo final:** A saída do labirinto que o agente deve alcançar, representada pela letra B.
- **Pontos de coleta:** Células obrigatórias marcadas como C_i, que o agente precisa visitar antes de ir para a saída na segunda etapa.
- **Regiões desconhecidas:** Células ocultas identificadas por ?, usadas apenas no modo de busca online.

2.3. Actuators (Atuadores)

Os atuadores são os mecanismos que permitem ao agente alterar sua posição no labirinto através de movimentos ortogonais (um passo por vez). O conjunto de ações válidas \$A\$ é composto por:

$$A = \{cima, baixo, esquerda, direita\}$$

2.4. Sensors (Sensores)

O funcionamento dos sensores do agente muda de acordo com o modo de execução do trabalho:

- **Modo de Busca Clássica (Offline):** O sensor faz a leitura completa do arquivo de texto do mapa. O agente conhece todo o labirinto antes de começar a calcular a rota.
- **Modo Busca Online:** O sensor funciona como um radar limitado com raio de percepção $r = 1$. O agente só consegue enxergar o que há nas quatro células vizinhas ao seu redor (cima, baixo, esquerda e direita), conforme as coordenadas:

$$S = \{(x - 1, y), (x + 1, y), (x, y - 1), (x, y + 1)\}$$

3. Classificação do Agente

O agente desenvolvido neste trabalho é classificado como um **Agente Baseado em Utilidade**.

Essa classificação se justifica porque o agente não busca apenas atingir um objetivo de forma binária (como um agente baseado em objetivos puro). Ele utiliza a função de desempenho J para medir a qualidade de diferentes caminhos possíveis, escolhendo a rota que oferece o melhor custo-benefício entre a distância percorrida e a quantidade de nós expandidos na memória.

Além disso, o agente é **baseado em modelo interno**. Essa característica é fundamental para a etapa de busca online, onde o agente não conhece o mapa inteiro e precisa salvar na memória as informações descobertas a cada passo para atualizar seu próprio mapa interno e planejar os próximos movimentos.

4. Formulação Formal do Problema

4.1. Busca Clássica no Labirinto Conhecido

O problema é formulado matematicamente pela tupla (S, A, T, s_0, G, c) , onde:

- **S**: conjunto de posições livres do labirinto (coordenadas vazias).
- **A**: conjunto de ações possíveis {cima, baixo, esquerda, direita}.
- **T**: função de transição (retorna a nova coordenada após a execução de um movimento válido).
- **s₀**: posição inicial A.
- **G**: teste de objetivo, isto é, alcançar a posição B.
- **c**: custo de cada movimento (custo unitário, $c = 1$).

4.2. Busca Local com Pontos de Coleta

O problema de busca local é modelado matematicamente pelos seguintes componentes:

- **4.2.1) Estado (s)**: Representa uma solução completa, definida como uma permutação (ordem específica) dos pontos de coleta a serem visitados. Para um cenário com 3 pontos, um estado possível é:

$$s = [C_1, C_2, C_3]$$

De forma geral, o estado é expresso $s = [C_{\pi(1)}, C_{\pi(2)}, \dots, C_{\pi(k)}]$ por, representa uma permutação dos índices.

- **4.2.2) Função de Custo (C(s))**: Avalia a qualidade do estado atual calculando a distância total da trajetória inteira (Início -> Pontos de Coleta na ordem de -> Saída). O objetivo do agente é minimizar a equação:

$$C(s) = d(A, C_{\pi(1)}) + \sum_{i=1}^{k-1} d(C_{\pi(i)}, C_{\pi(i+1)}) + d(C_{\pi(k)}, B)$$

onde $d(X, Y)$ é o custo do menor caminho válido entre dois pontos no labirinto (distância real calculada previamente através de um algoritmo de busca clássica, como o A*).

- **4.2.3) Vizinhaça e Ações (N(s))**: Define a estratégia de geração de novos estados (vizinhos) a partir do estado atual para a exploração do espaço de busca. A transição ocorre aplicando um operador de perturbação local, como o *swap* (troca de posição de dois pontos na sequência). O conjunto de vizinhos é formalizado por:

$$C(s) = d(A, C_{\pi(1)}) + \sum_{i=1}^{k-1} d(C_{\pi(i)}, C_{\pi(i+1)}) + d(C_{\pi(k)}, B)$$

4.3. Busca Online no Labirinto Desconhecido

O problema de busca online é formulado como (S, A, T, s_0, G, c, P) , onde:

- **S**: conjunto de posições livres do labirinto (coordenadas vazias).
- **A**: conjunto de ações possíveis {cima, baixo, esquerda, direita}.
- **T**: função de transição (determinística, mas parcialmente observável)
- **s₀**: posição inicial A.

- G: teste de objetivo, isto é, alcançar a posição B.
- c: custo de cada movimento.
- P: função de percepção(raio r).

Diferença da busca clássica: O agente não conhece T completamente. Ele descobre os resultados das ações apenas após executá-las ou percebê-las.

5. Descrição dos Algoritmos

5.1. Busca Clássica no Labirinto Conhecido

- **Busca em Largura (BFS):** Utiliza uma fila padrão (FIFO - *First-In, First-Out*). Expande sempre os nós mais rasos da árvore de busca primeiro. Garante o caminho ótimo em labirintos com custo de passo uniforme.
- **Busca em Profundidade (DFS):** Utiliza uma pilha (LIFO - *Last-In, First-Out*). Expande o nó mais profundo disponível na fronteira até encontrar um beco sem saída, para então retroceder. Não garante a rota ótima e pode gerar caminhos redundantes.
- **Busca de Custo Uniforme (UCS):** Utiliza uma fila de prioridade ordenada pelo custo real acumulado do ponto de partida até o nó atual, dado por $g(n)$. Como o labirinto possui custo unitário, sua execução é equivalente à da Busca em Largura.
- **Busca Gulosa:** Utiliza uma fila de prioridade ordenada exclusivamente pela função heurística $h(n)$ (distância geométrica até o objetivo). O algoritmo prioriza os nós que parecem estar mais próximos da saída, sendo rápido, mas sem garantia de otimização.
- **Busca A*:** Utiliza uma fila de prioridade que combina o custo real acumulado e a estimativa heurística, ordenada pela função $f(n) = g(n) + h(n)$. É um algoritmo completo e ótimo, encontrando o caminho mais curto de forma mais eficiente que a BFS ao evitar a expansão de nós em direções opostas ao objetivo.

5.2. Busca Local com Pontos de Coleta

- **Hill Climbing (Subida da Encosta):** Inicia com uma solução candidata aleatória, representada pela ordem de visita dos pontos de coleta. A cada iteração, gera uma solução vizinha por meio da troca ou inversão de posições dos *checkpoints* e compara seus custos. Caso a nova solução possua custo menor ou igual ao atual, ela é aceita. O algoritmo busca melhorar progressivamente a solução, porém pode ficar preso em ótimos locais, não garantindo a obtenção da melhor rota global.
- **Simulated Annealing (Recozimento Simulado):** Também inicia com uma solução aleatória e explora sua vizinhança por meio de modificações na ordem de visita dos pontos de coleta. Diferentemente do Hill Climbing, permite aceitar temporariamente soluções piores com uma probabilidade controlada pela temperatura do sistema. Conforme a temperatura diminui ao longo das iterações, a busca torna-se mais conservadora. Essa característica reduz a chance de aprisionamento em ótimos locais e aumenta a probabilidade de encontrar soluções próximas do ótimo global.
- **Função de Custo:** Avalia a qualidade de uma solução calculando a soma dos menores caminhos entre todos os pontos da sequência obrigatória de visita. O custo total é dado pela rota:

$$A \rightarrow C_{\pi(1)} \rightarrow C_{\pi(2)} \rightarrow \dots \rightarrow C_{\pi(k)} \rightarrow B$$

onde A representa o ponto inicial, B o objetivo final $C_{\pi(i)}$ e os pontos de coleta visitados em uma determinada ordem. Quanto menor o custo total, melhor é considerada a solução.

5.3. Busca Online no Labirinto Desconhecido

Replanning com A* (Opção A)

O agente mantém um mapa interno progressivamente atualizado. A cada passo:

1. Percebe células na vizinhança (raio r)
2. Atualiza o mapa interno com as percepções
3. Planeja executando A* no mapa parcial (células desconhecidas são tratadas como potencialmente livres — heurística otimista)
4. Age executando o próximo passo do plano
5. Se encontra uma parede inesperada, replaneja

Heurística: Distância de Manhattan $h(n) = |x_n - x_b| + |y_n - y_b|$

6. Metodologia Experimental

O projeto foi desenvolvido inteiramente na linguagem de programação Python. Para reproduzir os testes e executar as simulações no labirinto, é necessário possuir o seguinte conjunto de arquivos salvos no mesmo diretório:

- **Arquivos de mapa (.txt):** Os arquivos de texto puro que contêm o desenho em grade dos labirintos a serem testados (como lab1.txt, lab2.txt, lab3.txt, etc.).

A execução do sistema ocorre de forma direta: basta rodar o arquivo main.py através do terminal e, quando solicitado pelo programa, digitar o nome ou o caminho do arquivo de texto correspondente ao labirinto que se deseja resolver.

7. Resultados e Discussão

7.1. Desempenho da Busca Clássica (Mapa Conhecido)

Algoritmo	Custo g (n)	Passos	Expandidos	Pico Fronteira	Tempo (s)	Utilidade (J)
BFS	45.0	45	132	06	0.000298	-26.55
DFS	45.0	45	162	04	0.000343	-31.05
UCS	45.0	45	132	06	0.000334	-26.55
Gulosa	93.0	93	122	06	0.000344	-32.25
A*	45.0	45	91	06	0.000324	-20.40

7.2. Desempenho da Busca Local (com Pontos de Coleta)

Algoritmo	Melhor Custo (Passos)	Pior Custo (Passos)	Custo Médio (Passos)	Tempo Médio (s)	Sucesso (%)
Hill Climbing	168	168	168,00	0,000537	100
Simulated Annealing	168	168	168,00	0,000599	100

7.3 Desempenho da Busca Online (Labirinto Desconhecido)

Replanning A*

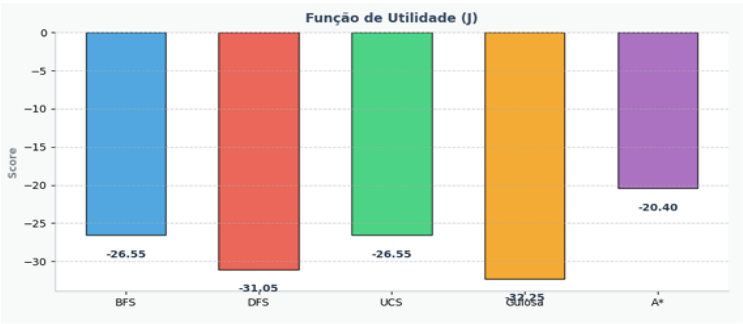
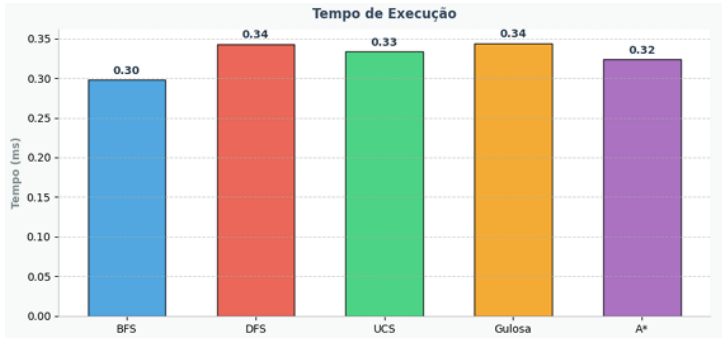
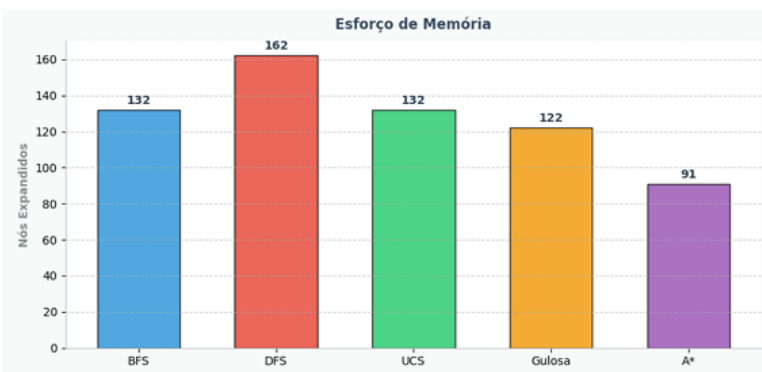
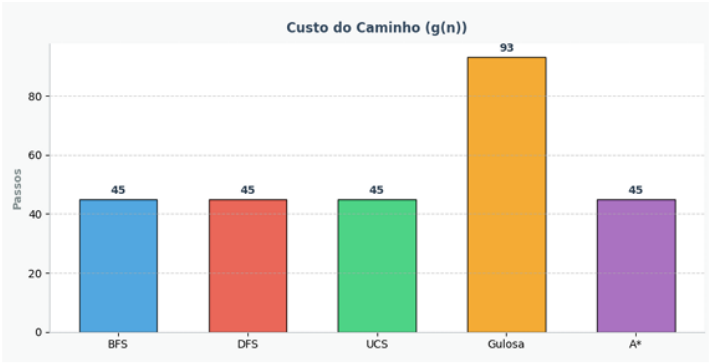
Raio	Custo Real	Revisitadas	Replanejamentos	Razão online/Offline
1	35	1	14	1.0606
2	35	1	8	1.0606
3	35	1	7	1.0606

Online DFS

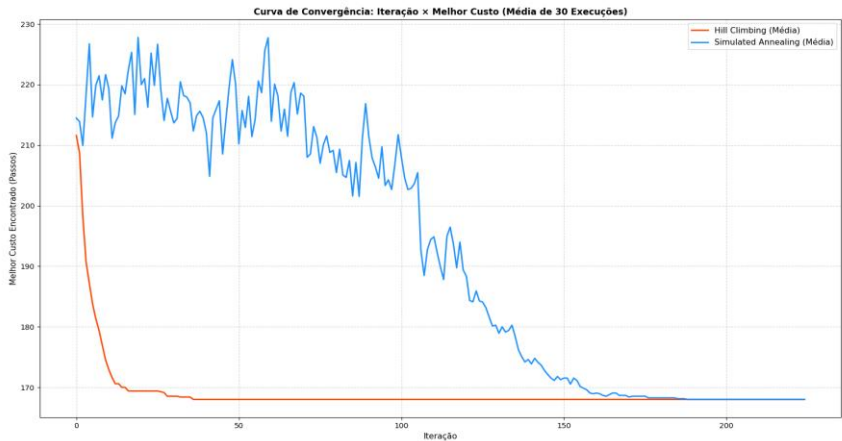
Raio	Custo Real	Revisitadas	Replanejamentos	Razão Online/Offline
1	297	116	297	9.000
2	297	116	297	9.000
3	297	116	297	9.000

8.Gráficos

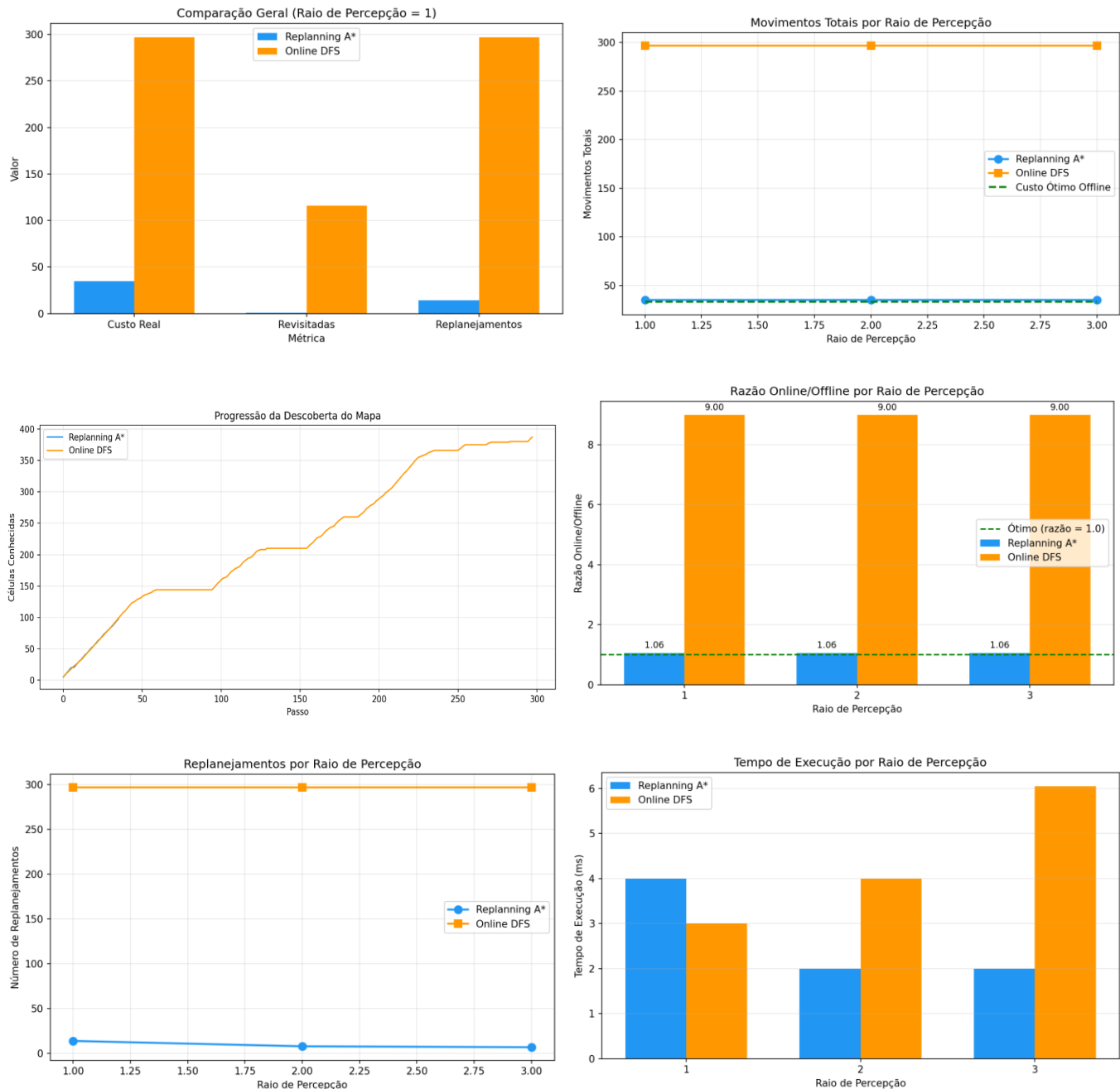
8.1. Busca Clássica



8.2. Busca Local



8.3. Busca Online



9. Análise Crítica

9.1. Busca Clássica

9.1.1. Sim. A Busca em Largura (BFS) encontrou o caminho ótimo com custo de 45 passos.

Condição de ocorrência: Isso ocorre estritamente porque o ambiente possui custo de passo uniforme (mover-se para qualquer célula vizinha custa $\$c=1\$$). Nessas condições, a expansão radial em "ondas" do BFS garante que o primeiro caminho a interceptar o objetivo seja, de forma absoluta, o mais curto possível.

9.1.2. Não foi o mais rápido. O DFS expandiu 162 nós, configurando o maior esforço computacional e de varredura de toda a tabela, refletindo em um dos maiores tempos (0,000343 s).

A solução: Neste mapa específico (Labirinto 1), o DFS obteve uma solução ótima (custo 45). Contudo, isso foi uma anomalia causada pela topologia linear do labirinto que canalizou a busca. O algoritmo em si não possui heurística ou mecanismo de otimização, e geralmente produz rotas muito mais longas e tortuosas em cenários abertos.

9.1.3. Não diferiu, pois, os custos do ambiente não variaram: No domínio testado, todas as células livres possuem o mesmo custo de travessia ($\$c=1\$$). Por causa disso, a Busca de Custo Uniforme (UCS) se

degradou em uma BFS, expandindo a mesma quantidade de nós (132) e obtendo o mesmo custo (45). A UCS só demonstraria diferença e superioridade em um mapa com relevos de pesos diferentes (ex: asfalto custando 1 e lama custando 5).

9.1.4. Foi moderadamente eficiente, mas fortemente subótimas. Ao ser atraída geograficamente para o alvo, expandiu menos nós que a BFS e a UCS (122 vs 132). Otimalidade: Falhou severamente em encontrar o menor caminho, retornando a pior rota do experimento com custo de 93 passos (mais que o dobro do ideal). A miopia do algoritmo fez com que ele avançasse cegamente para o objetivo, caindo em armadilhas de obstáculos (paredes em "U") e sendo forçado a dar grandes voltas para escapar.

9.1.5. Sim, de forma ideal (Melhor Utilidade $J = -20.40$): O A^* obteve o melhor dos dois mundos. Ele garantiu a otimalidade absoluta do trajeto (custo 45, igual ao da BFS), mas fez isso expandindo o menor número de nós de todo o laboratório (apenas 91 nós). Ao somar o custo real com a estimativa heurística ($f(n) = g(n) + h(n)$), ele evitou a exploração inútil na direção oposta ao objetivo sem perder a garantia matemática da rota mais curta.

9.1.6. Sim, a heurística é plenamente admissível. A propriedade de admissibilidade exige que a estimativa nunca seja maior que o custo real ($h(n) \leq h^*(n)$). Em um labirinto onde o agente só se move ortogonalmente, a Distância de Manhattan calcula os passos necessários assumindo um mapa totalmente vazio. Como a presença de paredes de colisão no ambiente real apenas obriga o agente a desviar e dar mais passos, o custo real sempre será maior ou igual à estimativa da heurística. Nunca haverá superestimação.

9.2. Busca Local

9.2.1. Não. Os dados mostram que o Hill-Climbing obteve sucesso nas 30 execuções, encontrando sempre o melhor custo de 168 passos.

9.2.2. Não, os dois algoritmos garantiram a melhor rota sequencial estabelecida como ideal seguindo o percurso A, C2, C3, C1 e B. A variação observada restringiu-se ao comportamento dinâmico durante a execução, não ao resultado final.

9.2.3. Foi observado através da flutuação inicial no gráfico, a aceitação temporária de soluções menos eficientes, mostrando a variação de passos do algoritmo antes da fase de refinamento.

9.2.4. O coeficiente de resfriamento estabelecido em 0.95 fez com que a convergência fosse mais rápida.

9.2.5. Sim, conforme as métricas, foi vista a uniformidade entre os custos máximo e mínimo obtidos, pior custo encontrado: 168 e melhor custo encontrado: 168.

9.2.6. Os testes comprovaram o compromisso entre tempo e qualidade ao manipularmos os limites de execução: focando no tempo do Simulated Annealing para um resfriamento brusco (encerrando em médias de 17 iterações), a execução foi extremamente rápida ($\sim 0.000075s$), mas a qualidade caiu para 63.33% de sucesso, frequentemente travando em rotas subótimas de até 186 passos. Em contrapartida, ao focarmos na qualidade configurando o Hill Climbing para explorar detalhadamente por 500 iterações, garantimos 100% de sucesso na rota perfeita de 168 passos, mas o compromisso exigido foi um tempo de processamento significativamente maior ($\sim 0.001830s$). Fica evidente que economizar tempo computacional faz o algoritmo parar prematuramente em mínimos locais, enquanto o investimento de tempo é essencial para assegurar a descoberta da solução ótima global.

9.3. Busca Online

9.3.1. Replanning A^* : Tomou decisões ligeiramente subótimas (razão 1.06). Isso ocorre porque, com mapa parcialmente conhecido, o agente pode iniciar um caminho que depois se revela bloqueado, forçando um desvio. O custo extra de 2 movimentos (35 vs 33) representa apenas um pequeno "pedágio" pela falta de informação completa.

9.3.2. A disposição completa de paredes no labirinto, atalhos e caminhos alternativos, becos sem saída (que o agente só descobre ao entrar neles) e a topologia global do labirinto

9.3.3. Replanning A*: O mapa interno convergiu parcialmente — apenas as regiões ao longo do caminho percorrido foram reveladas (110 de ~425 células livres com raio 1). O agente não precisa conhecer o mapa inteiro para encontrar o objetivo.

9.3.4. Replanning A*: Praticamente não revisitou (apenas 1 célula revisitada). A heurística A* direciona eficientemente o agente.

9.3.5. Aumentar o raio de percepção (reduz replanejamentos de 14 para 7 no A*), usar D Lite* para replanning incremental (evita recalcular A* do zero), memorizar becos sem saída para evitar reentrada e exploração dirigida priorizando fronteiras com informação desconhecida

9.3.6. O que diferencia busca online de busca clássica?

Aspecto	Busca Clássica	Busca Online
Conhecimento	Mapa completo	Mapa parcial
Planejamento	Uma vez, antes de agir	Contínuo, a cada passo
Garantia de otimalidade	Sim (A*)	Não
Custo computacional	Baixo (1 execução)	Alto (Múltiplos replanejamentos)
Adequação	Ambientes estáticos conhecidos	Ambientes desconhecidos/dinâmicos

10. Limitações

10.1. Busca Clássica

- **Busca em Largura (BFS):** Consumo excessivo de memória. Em mapas abertos e grandes, a fronteira cresce exponencialmente, podendo esgotar a RAM.
- **Busca em Profundidade (DFS):** Não garante a melhor rota. Pode gerar caminhos extremamente longos, redundantes e ineficientes se escolher a direção errada no início.
- **Busca de Custo Uniforme (UCS):** É inútil em labirintos onde todo passo tem o mesmo custo ($c=1$), pois gasta o mesmo tempo e memória que o BFS sem trazer nenhuma vantagem.
- **Busca Gulosa:** É "miope". Como só olha para o objetivo final, costuma ficar presa em paredes com formato de "U", sendo forçada a dar voltas imensas e gerando rotas muito ruins.
- **Busca A*:** Assim como o BFS, guarda todos os caminhos na memória, o que pesa em mapas gigantes. Além disso, seu sucesso depende totalmente de uma boa heurística.

10.2. Busca Local

- **Hill Climbing:** Fica preso em "ótimos locais". Se a rota precisar piorar temporariamente para depois melhorar de verdade, ele congela e entrega um resultado ruim.
- **Simulated Annealing:** É extremamente sensível às configurações (temperatura e resfriamento). Se esfriar rápido demais, falha igual ao Hill Climbing; se esfriar devagar, demora muito para rodar.

10.3. Busca Online

1. **Heurística otimista no A* parcial:** Tratar células desconhecidas como livres pode levar a caminhos que se revelam bloqueados.
2. **Custo computacional:** Replanejamento frequente com A* pode ser custoso em labirintos muito grandes.
3. **DFS sem heurística:** O Online DFS não utiliza informação de direção, resultando em exploração ineficiente.
4. **Raio de percepção fixo:** Em cenários reais, a percepção pode variar com obstáculos.

11. Uso de IA

Este trabalho utilizou o *Google Gemini* e o *GitHub Copilot (Claude Opus 4)* como ferramentas de apoio em todas as etapas do projeto. O uso da Inteligência Artificial se dividiu em quatro frentes principais:

Embasamento Teórico: Esclarecimento de conceitos matemáticos cruciais (como Pico de Fronteira e Efeito Corredor), tradução de jargões de otimização combinatória e definição da lógica de percepção para a Busca Online.

Desenvolvimento e Refatoração: Geração rápida dos dashboards comparativos (usando Matplotlib), criação da interface gráfica iterativa (Pygame), rotinas de exportação para CSV e refatoração da Busca Local para rodar em cenários de estresse.

Depuração (Debugging): Identificação e correção de um `IndexError` na leitura do labirinto (Claude) e resolução de bugs de formatação visual na elaboração do documento (Gemini).

Análise de Dados e Redação: Auxílio na interpretação estatística dos logs do terminal (taxas de sucesso, estabilização de hiperparâmetros) e lapidação do texto final das análises críticas e conclusões.

Nota de Validação: Nenhuma sugestão estrutural das IAs foi rejeitada, porém, 100% dos resultados teóricos e práticos (como custos de caminho e razão de subotimalidade) foram testados e validados manualmente pela equipe para garantir a integridade acadêmica do trabalho.

12. Conclusão

Este trabalho prático demonstrou, na prática, como diferentes filosofias de Inteligência Artificial resolvem exatamente o mesmo problema. Ao testar o agente no labirinto, ficou claro que fornecer mais informações e uma boa estratégia matemática muda completamente o desempenho da máquina e o gasto de memória.

Na etapa de Busca Clássica (mapa conhecido), o algoritmo A^* provou ser absoluto. Ele conseguiu o equilíbrio perfeito: encontrou o caminho mais curto sem desperdiçar memória procurando em lugares inúteis, algo que a Busca em Largura (BFS) e a Busca em Profundidade (DFS) não conseguiram fazer tão bem. Já na Busca Online, onde o mapa era desconhecido, vimos que a falta de visão cobra um preço alto. Sem saber o que há pela frente, o agente é forçado a bater em paredes e andar em círculos, provando que agir no escuro é muito mais custoso do que planejar offline.

Por fim, no desafio de otimizar a rota dos pontos de coleta (Busca Local), o algoritmo de Têmpera Simulada (Simulated Annealing) mostrou sua superioridade em relação ao Hill Climbing. Ao permitir "pioras temporárias" na rota, ele conseguiu escapar de armadilhas matemáticas e convergiu para o melhor caminho muito mais rápido e com menos esforço computacional.

Em resumo, a principal lição deste laboratório é que não existe um "algoritmo perfeito" para todas as situações. O segredo de uma boa IA não é apenas encontrar a saída, mas saber escolher a ferramenta certa dependendo do quanto você conhece do ambiente e de quanta memória o seu computador tem disponível.